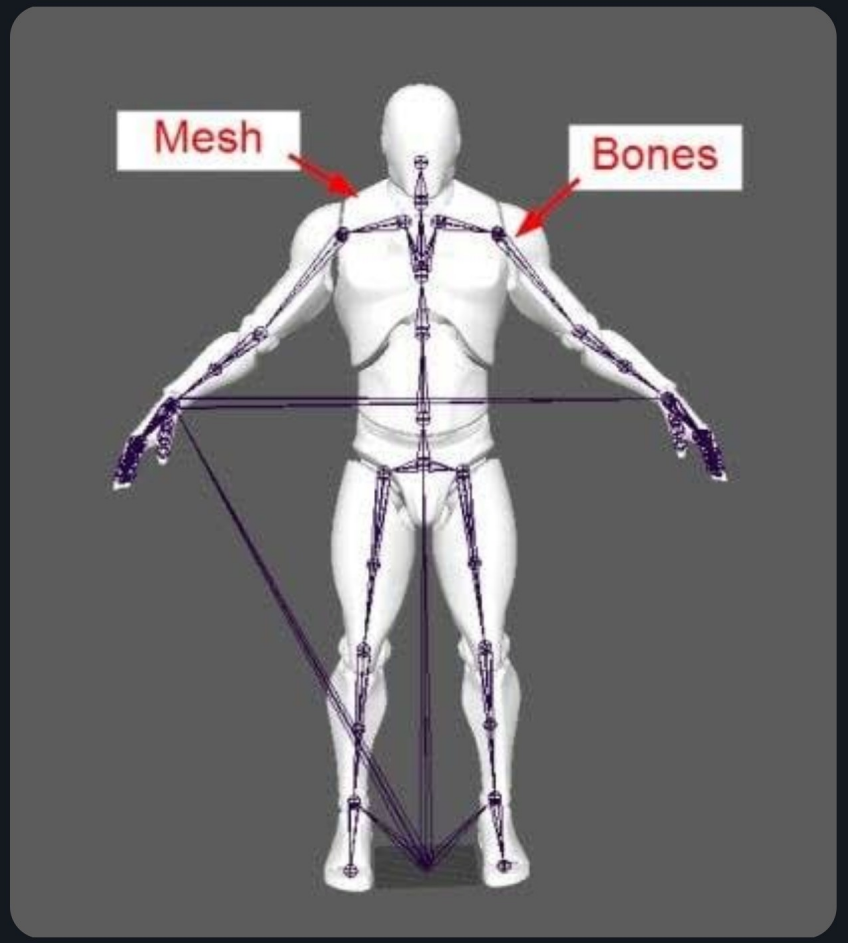
GAME ENGINE ANIMATION STUDY

Animation System

애니메이션의 종류 · Skeletons · Poses · Clips

[발표자 이름] · 2026.10



AGENDA

오늘 다룰 내용

01

애니메이션의 종류

Cel부터 Skinned Animation까지, 캐릭터를 움직이는 방식의 변천

02

Skeletons

관절의 부모-자식 계층 구조와 메모리 속 표현

03

Poses

Bind Pose · Local Pose · Global Pose

04

Clips

포즈를 시간축에 놓은 애니메이션 클립과 그 데이터

캐릭터를 움직이는 네 가지 방식

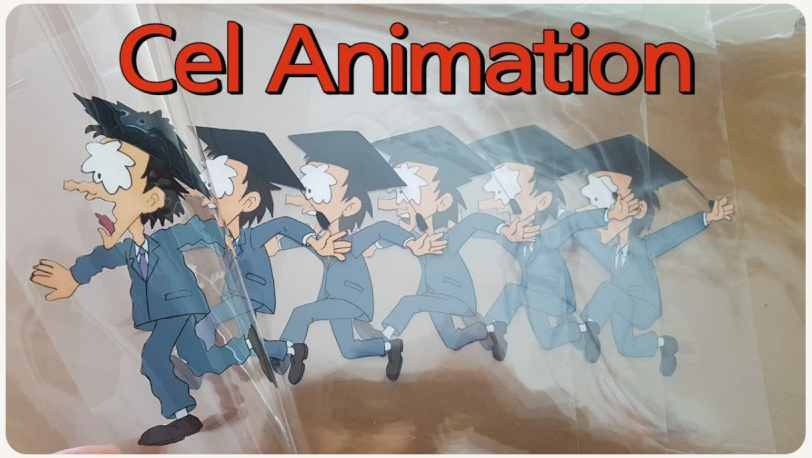
1-1	**Cel · Sprite**	프레임 이미지를 빠르게 교체해 움직임을 표현 (2D)
1-2	**Rigid Hierarchical**	단단한 조각을 계층으로 연결 · 관절이 갈라지는 문제
1-3	**Per-Vertex · Morph Target**	정점을 직접 움직여 자연스러운 변형 · 데이터가 큼
1-4	**Skinned Animation**	뼈대에 피부 메쉬를 바인딩, 가중치로 자연스럽게 변형 → 오늘의 주제

셀 애니메이션 (Cel Animation)

> 프레임을 빠르게 전환해 움직이는 것처럼 보이게 하는 애니메이션

> 전통 만화 방식: 배경은 그대로 두고, 움직이는 부분만 연속 이미지로 교체

> 게임에서는 **Sprite Animation**으로 이어져 2D 애니메이션에 사용



Cel Animation



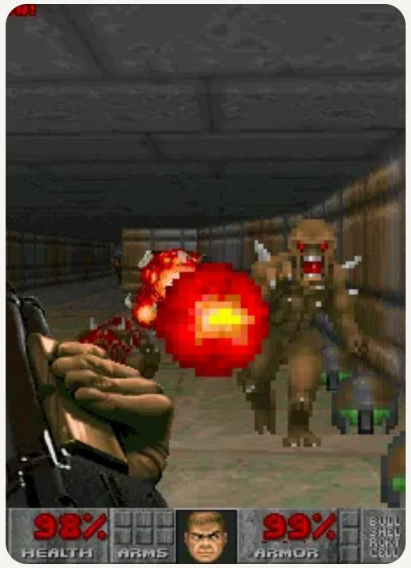
Sprite Sheet

강체 계층구조 애니메이션

> 초기 3D 게임은 카메라를 향한 Quad에 이미지를 붙인 방식 → 지금도 관중, 원거리 군대 같은 저해상도·원거리 오브젝트 최적화에 사용

> 이후 캐릭터를 단단한 조각(Rigid Pieces)의 집합으로 모델링하고, 조각들이 계층적으로 서로 구속(Constraint)됨

> **문제: 관절 부위 갈라짐 (Cracking)**



초기 3D · 스프라이트



바이오하자드 1



마인크래프트

Per-Vertex Animation

강체는 덩어리처럼 움직여 부자연스럽다 → 피부·근육이 늘어나듯 **개별 vertex를 직접** 움직인다 (Point Cache, Alembic)



방식	Bone 없이 정점이 매 프레임 어디로 이동하는지 기록 (브루트 포스)
장점	구상한 모든 메쉬 변형을 세밀하게 표현
단점	모든 버텍스의 시간별 모션을 저장 → 데이터 집약적, 용량이 큼

1-3 · 모프 타겟

Morph Target Animation

1 **극단적 포즈 제작** — 표정 없는 기본 모델을 복사해 토폴로지는 그대로, 정점 위치만 옮겨 소수의 Extreme Pose를 만든다. (기준 위치 + 차이값)으로 저장

2 **실행 시점에 혼합** — 고정된 포즈 두 개 이상을 섞어 애니메이션을 만든다

3 **선형 보간** — 각 버텍스 위치는 극단 포즈들 사이의 선형 보간. 0.0 ~ 1.0 슬라이더로 실시간 계산

버텍스 위치 계산

$$V = V_0 + \Sigma W_i \cdot \Delta_i$$

V_0 : 기본 메쉬의 정점 위치
Δ_i : i번째 타겟과의 차이값
W_i : 0.0 ~ 1.0 가중치 (슬라이더)

얼굴 애니메이션과 하이브리드 리그

> 얼굴은 약 50개 근육이 움직이는 복잡한 구조 → 모든 버텍스를 직접 제어하는 모프 타겟이 적합

> 대안: 해부학적 근육 구조를 바탕으로 한, 수백 개 관절의 **Jointed Facial Rig**

> **하이브리드**: 관절 리그로 주요 포즈를 잡고, 모프 타겟으로 미세 조정

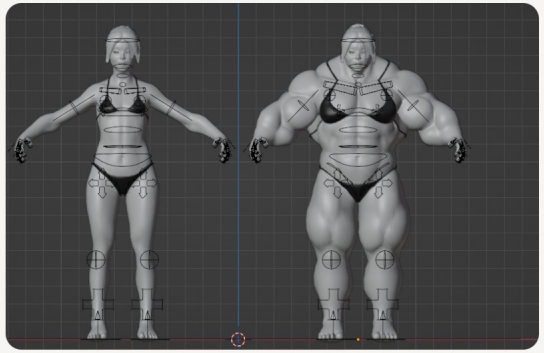
하이브리드 결합 방식의 작동 원리

1. **뼈대(Joint Rig)로 큰 움직임(Macro) 제어:**
   - 턱을 벌리기, 고개를 돌리기, 입술을 크게 우그러뜨리기, 목의 움직임 등 **뼈대의 회전과 이동이 효율적인 큰 동작들**은 수백 개의 관절 리그가 담당합니다. (데이터 압축률이 좋고 일관된 애니메이션 유지에 유리합니다.)
2. **몰프 타겟(Morph Target)으로 미세 디테일(Micro) 보완:**
   - 관절만으로는 표현하기 힘든 **피부의 미세한 주름, 입술이 맞닿을 때의 찌그러짐, 볼의 미세한 떨림, 눈가 주름** 등은 몰프 타겟(BlendShape)을 덧씌워서 해결합니다.
3. **최종 렌더링 시 합산 (Additive Blending):**
   - 게임 엔진 안에서는 이 두 가지 변형 값이 실시간으로 더해집니다.
   - $$\text{최종 얼굴} = \text{기본 모델} + \text{얼굴 뼈대 움직임(Joints)} + \text{미세 보정 델타값(M}$$

1-3 · 모프 타겟의 활용

Shape Target · Expression Target

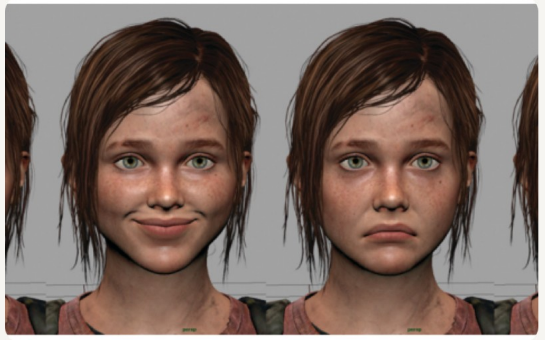
Shape Target — 얼굴형, 코 높이·모양 등 형태 변경 · 체형도 가능





Expression Target — 표정용, 0~1 값으로 동작





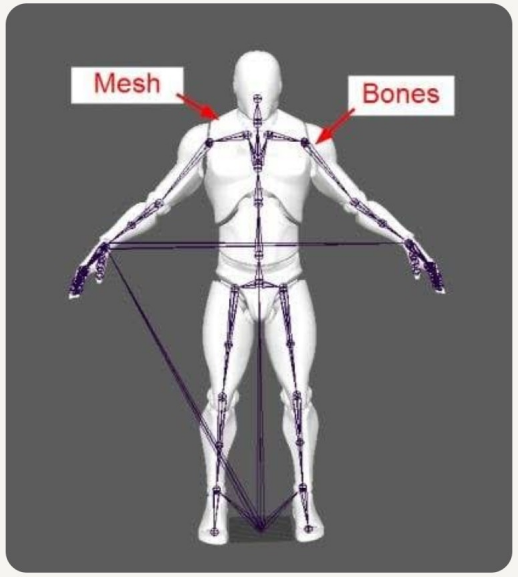
오른쪽: The Last of Us

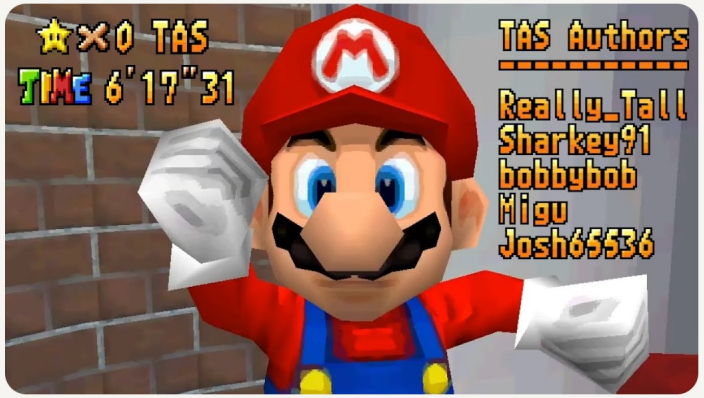
프로젝트 회고

체형은 bone 길이까지 함께 바뀌는 경우가 있다 — 우리 프로젝트에서 잘 안 됐던 건 player bone이 아니라 cutscene용 캐릭터의 bone을 그대로 가져왔기 때문일지도

Skinned Animation

- 강체 계층구조보다 효율적인 성능·메모리로 피부와 옷의 움직임을 사실적으로 표현 · 영화와 게임 모두 사용
- Bone은 숨기고, Skin이라 불리는 연속 삼각형 메쉬를 관절에 바인딩 → Vertex가 관절의 움직임을 추적
- 한 정점이 여러 관절의 weight를 받아 Skin이 자연스럽게 늘어남





2 · SKELETONS

관절마다 부모 인덱스 하나만 저장하면, 뼈대 전체가 표현된다

Root의 부모 인덱스는 −1 같은 무효값으로 처리

관절은 부모-자식 트리를 이룬다

> **Root Joint**: 계층의 최상위에 있는 단 하나의 근본 관절 (예: 골반 / Pelvis)

> **부모-자식 관계**: 모든 자식 관절은 단 하나의 부모 관절만 가진다

> 그래서 각 관절에 **부모 관절의 인덱스**만 저장하면 전체 뼈대 구조를 완전히 표현할 수 있다 (Root의 부모 = −1 같은 무효값)

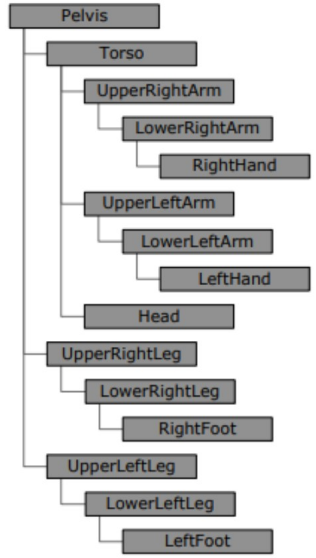


Figure 13.2. A typical skeletal hierarchy for a bipedal character.

메모리 속 뼈대: 관절 배열

> 엔진 메모리에서는 전체 관절 데이터를 **배열(Array)**로 저장해 가볍고 효율적으로 표현

> 부모 관절이 배열에서 항상 자식보다 먼저 오도록 정렬 → **Index 0은 항상 Root**

> Skinned Mesh의 정점도 인덱스로 결합된 joint를 참조

Index	Joint	Parent
0	Pelvis (Root)	−1
1	Torso	0
2	UpperRightArm	1
3	LowerRightArm	2
4	RightHand	3
…	…	…
12	UpperLeftLeg	0

Joint와 Skeleton 구조체

```
struct Joint
{
    Matrix4x3 m_invBindPose; // 역 바인드 포즈 변환
    const char* m_name;      // 관절 이름
    U16 m_iParent;           // 부모 인덱스 (루
트: 0xFFFFu)
};

struct Skeleton
{
    U32 m_jointCount;        // 관절의 개수
    Joint* m_aJoint;         // 관절들의 배열
};
```

m_invBindPose

> 피부 메쉬가 뼈대에 처음 결합된 시점(Bind Pose)의 위치·회전·크기 정보의 **역행렬**

> 실시간 연산 시 버텍스를 뼈의 로컬 공간으로 가져왔다가 현재 애니메이션 위치로 변환할 때 필수

3 · Poses

Pose = 관절을 회전·이동·크기 조절한 상태

Joint Pose

특정 기준 좌표계에 대한 관절의 위치 · 방향 · 크기

4×4 / 4×3 행렬 또는 SRT

Skeleton Pose

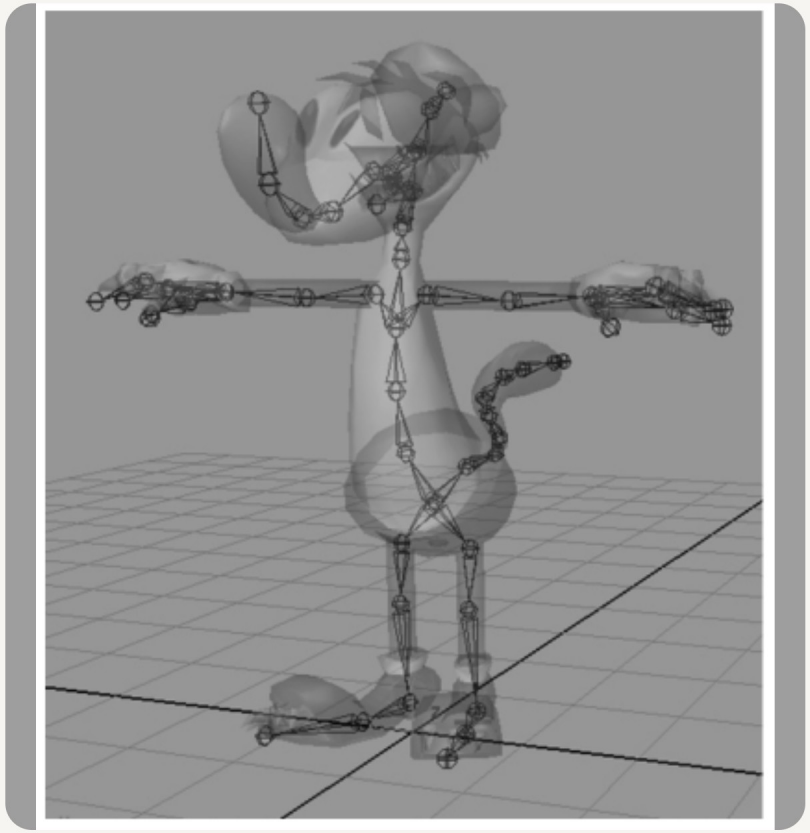
모든 Joint Pose의 집합

행렬 또는 SRT의 단순 배열

이어서 볼 세 가지 Pose **Bind Pose** **Local Pose** **Global Pose**

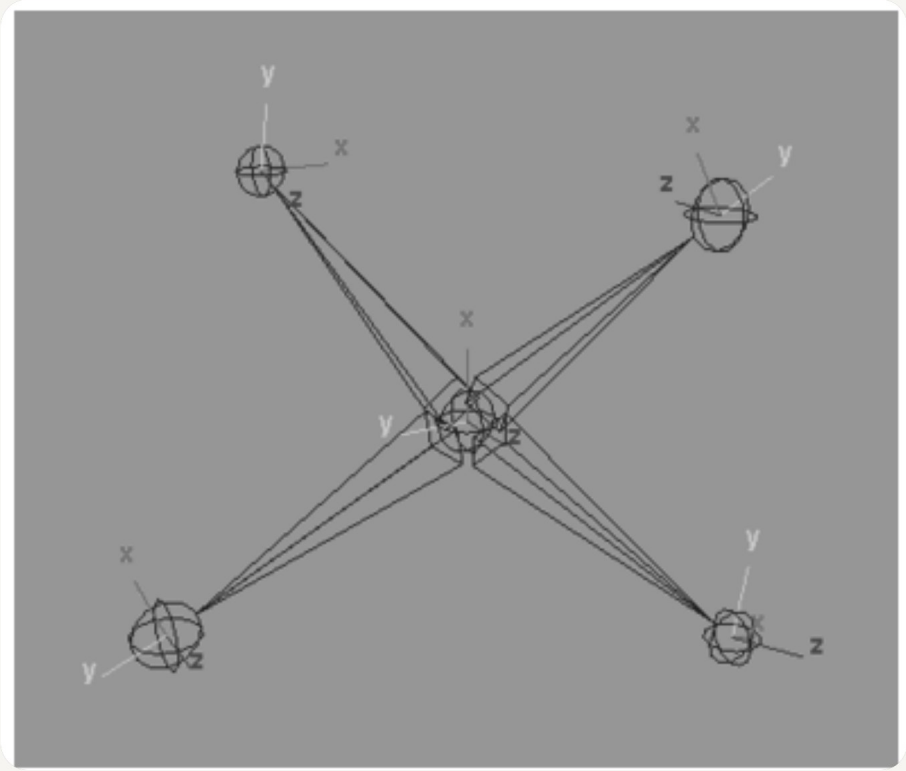
Bind Pose (Reference · Rest Pose)

- 3D 메시를 Skeleton에 바인딩하기 전의 포즈
- Skeleton 없이 일반 비스키닝 삼각형 메시로 렌더링할 때의 모습
- 보통 T자나 A자로 만들어 **T-Pose**, **A-Pose**라고 부른다



Local Pose: 부모를 기준으로

- 부모를 기준으로 정의한 Pose를 **Local Pose**라고 한다
- Joint Pose는 대부분 부모 Joint를 기준으로 지정한다
- 어깨 Joint를 회전시키면 팔꿈치, 손목, 손가락도 연결된 채 함께 회전한다
- Local Pose는 대부분 **SRT 형식**으로 저장 → 블렌딩에서 필요



Joint Scale: 엔진마다 스케일을 다르게 제한한다

가정	표현	float 수	비고
스케일 없음	S_j 생략 → 단위행렬로 간주	0	관절 크기를 절대 바꾸지 않음
균일 스케일	스칼라 s_j 하나	1	세 차원 모두 같은 비율
비균일 스케일	$s_j = [sx_j, sy_j, sz_j]$	3	3×3 행렬 S_j의 대각 원소 → 엄밀한 벡터는 아님
스케일·전단	완전한 3×3 행렬 S_j	9	가능하지만 게임 엔진에서는 드묾

메모리 절약

애니메이션 프레임마다, 관절마다 저장할 값이 줄어든다

균일 스케일 → 계산이 단순

경계구가 타원체로 바뀌지 않아 관절 단위 절두체·충돌 검사가 쉬워진다

Joint Scale in Unity · Unreal

Unity

> 뼈 = Transform, **localScale은 Vector3** → 비균일 스케일 허용

> Generic 리그는 뼈 스케일 애니메이션 재생 가능, **Humanoid 리그는 미지원** (변환 시 경고)

> 회전된 자식 + 비균일 스케일 부모 → **skew 발생**, lossyScale은 근사값

Unreal

> 뼈 = **FTransform** (FQuat 회전 · FVector 이동 · FVector Scale3D) → SRT

> 비균일 스케일 애니메이션 지원, **스케일이 1이 아닐 때만 저장**해 메모리 절약

> SRT 합성이라 **전단을 표현하지 않음** → 자식이 기울지 않음

Local Pose의 SRT 표현

```
struct JointPose
{
    Quaternion m_rot;   // R
    Vector3    m_trans; // T
    F32        m_scale; // S, 균일 스케일만 허용
};
```

```
struct JointPose
{
    Quaternion m_rot;   // R
    Vector4    m_trans; // T
    Vector4    m_scale; // S, 비균일 스케일 허용
};
```

```
struct SkeletonPose
{
    Skeleton*  m_pSkeleton;  // 골격 및 관절 수
    JointPose* m_aLocalPose; // 로컬 관절 포즈
};
```

$$\mathbf{P}_j = \begin{bmatrix} \mathbf{S}_j\mathbf{R}_j & \mathbf{0} \\ \mathbf{T}_j & 1 \end{bmatrix},$$

$$\mathbf{P}^{\text{skel}} = \left\{ \mathbf{P}_j \right\} \Big|_{j=0}^{N-1}.$$

로컬 포즈 = 자식 공간 → 부모 공간 변환

로컬 Joint Pose는 바로 위 부모가 기준 → 모든 아핀 변환은 점·벡터를 한 좌표 공간에서 다른 좌표 공간으로 옮기는 것으로 볼 수 있다

관절 j 공간의 점·벡터에 P_j를 적용하면 같은 점·벡터가 **부모 관절 공간**으로 표현된다 (p(j): 부모 인덱스)

$$P_j = (P_{C \to P})_j = P_{j \to p(j)}$$

반대로 부모 → 자식 공간 변환은 로컬 관절 포즈의 **역변환**

$$P_{p(j) \to j} = [\ P_{j \to p(j)}\ ]^{-1}$$

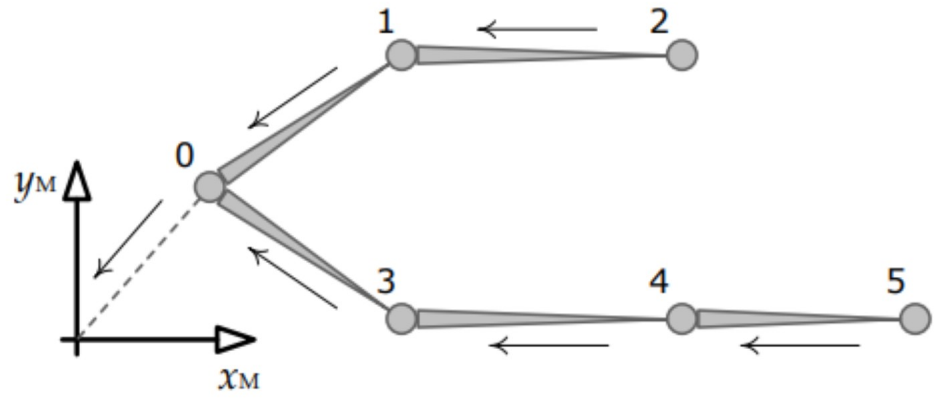
Global Pose: 모델·월드 공간 기준

> Joint Pose를 모델 공간이나 월드 공간으로 표현하면 편리할 때가 있다 → 이것이 **Global Pose**

> 엔진에 따라 행렬로 표현하기도, SRT 형식을 쓰기도 한다

```
struct SkeletonPose
{
    Skeleton*  m_pSkeleton;  // 골격 및 관절 수
    JointPose* m_aLocalPose; // 로컬 관절 포즈
    Matrix44*  m_aGlobalPose; // 글로벌 관절 포즈
};
```

Global Pose 구하기: Root까지 차례로 곱한다



$$\mathbf{P}_{2\to M} = \mathbf{P}_{2\to 1}\mathbf{P}_{1\to 0}\mathbf{P}_{0\to M}.$$

$$\mathbf{P}_{5\to M} = \mathbf{P}_{5\to 4}\mathbf{P}_{4\to 3}\mathbf{P}_{3\to 0}\mathbf{P}_{0\to M}.$$

일반식

$$\mathbf{P}_{j\to M} = \prod_{i=j}^{0} \mathbf{P}_{i\to p(i)},$$

4 · Clips

게임 애니메이션은 작은 클립 수천 개로 나뉜다

영화

장면의 모든 요소와 카메라를 미리 계획 → 장면 전체를 길고 연속적인 프레임 시퀀스로 제작

게임

상호작용 때문에 움직임을 예측할 수 없음 → 작은 동작 단위인 **애니메이션 클립**으로 분할

반복: 걷기 · 달리기 사이클 **1회: 던지기 · 넘어지기** **전신: 점프** **일부: 오른팔 흔들기**

예외: IGC · NIS · FMV 같은 비상호작용 구간은 영화처럼 제작, QTE는 반상호작용

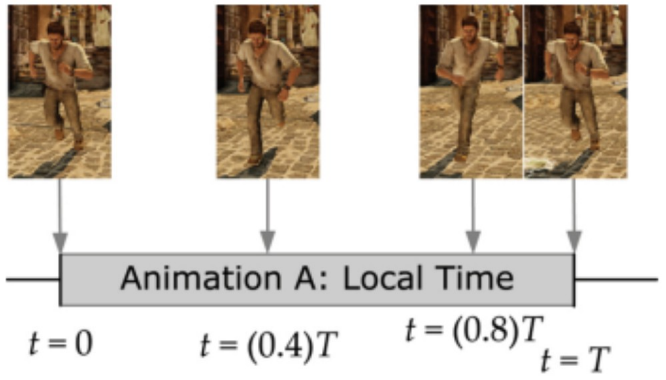
4-1 · 로컬 타임라인

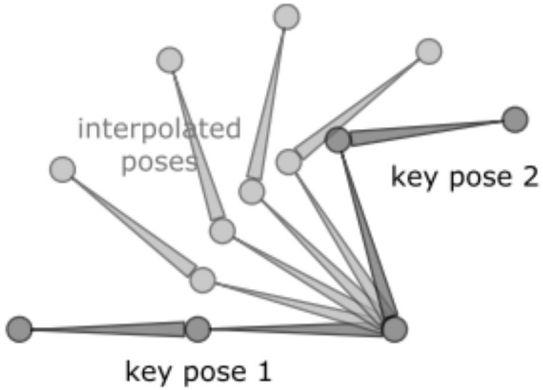
로컬 타임라인과 포즈 보간

> 클립마다 로컬 타임라인 t가 있다: 시작 t = 0, 끝 t = T (지속 시간)

> 애니메이터는 **키 포즈**만 만들고, 엔진이 선형·곡선 보간으로 사이 포즈를 채운다

> 그래서 t는 **실수** — 프레임 속도 변화·시간 배율 때문에 1.1, 1.9, 3.2…처럼 샘플링 (음수면 역재생)





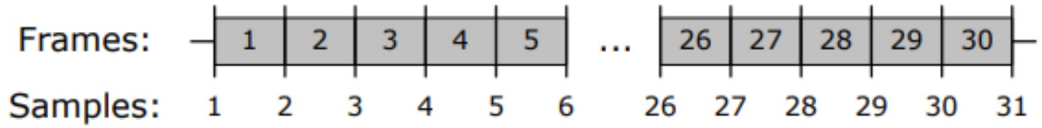
프레임 vs 샘플, 그리고 반복 클립

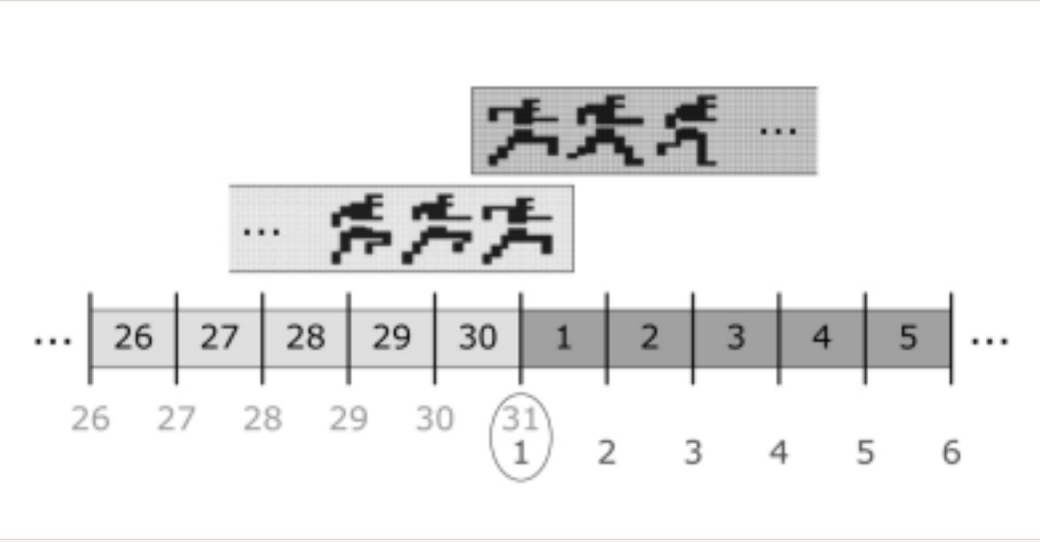
> **프레임** = 1/30·1/60초 길이의 구간, **샘플** = 한 시점

> 30fps 1초 클립 = 30프레임, 31샘플

> 반복 클립은 마지막 샘플이 다음 반복의 첫 샘플과 겹친다
> → 중복이라 보통 생략

N프레임 클립	고유 샘플 수
반복하지 않음	N + 1
반복	N





4-1 · 로컬 타임라인

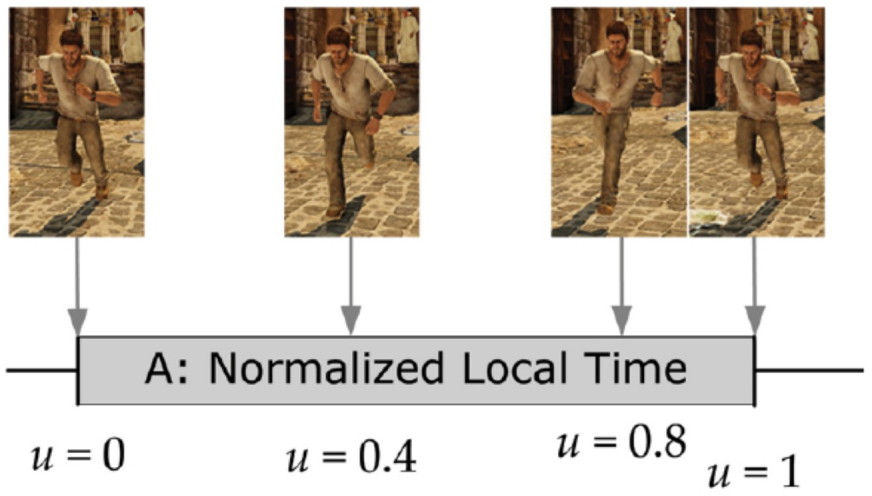
정규화 시간 (위상, Phase)

> 지속 시간 T와 관계없이 시작 **u = 0**, 끝 **u = 1**

> 반복할 때 사인파의 위상처럼 동작해서 위상(phase)이라고도 부른다

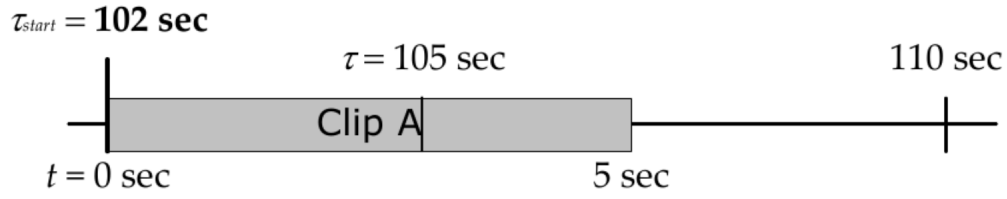
예: 길이가 다른 클립 동기화

달리기(2초) → 걷기(3초) 크로스페이드: u_walk를 u_run에 맞추고 같은 정규화 속도로 진행 → 발 위치가 계속 맞는다

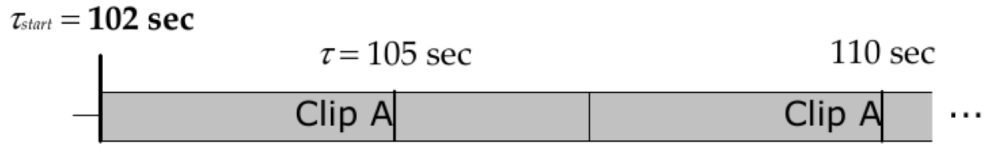


클립을 캐릭터의 글로벌 타임라인에 놓는다

- 캐릭터마다 **글로벌 시간 τ**가 흐른다 (생성 또는 레벨 시작부터)
- 재생 = 클립의 로컬 타임라인을 글로벌 타임라인에 대응시키는 것
- 반복 재생 = 클립 복사본을 앞뒤로 이어 붙이는 것



τ_start = 102초에서 클립 A 재생 시작

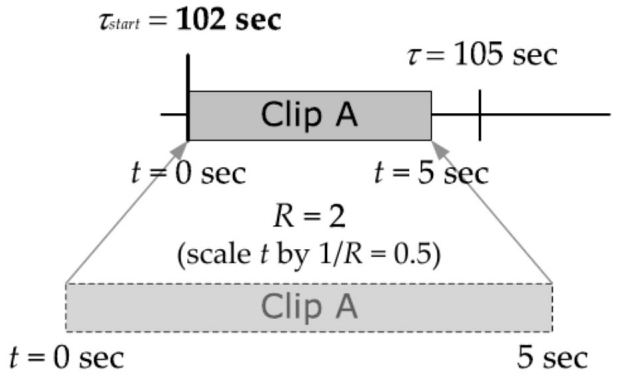


반복 재생: 복사본을 이어 붙임

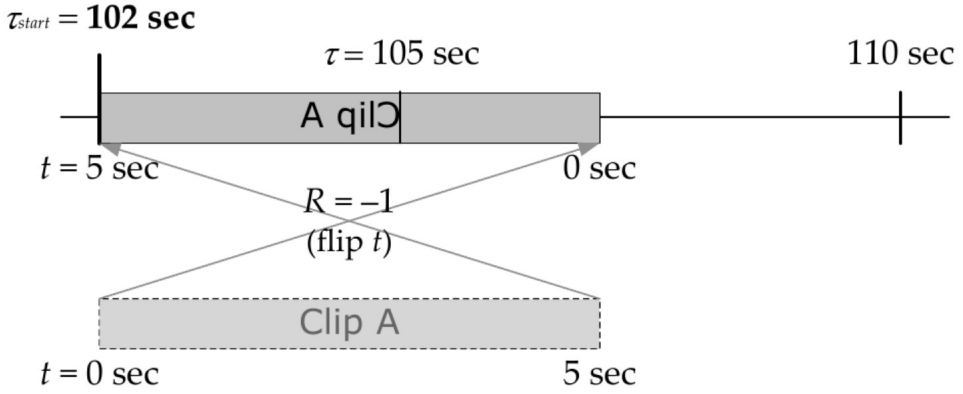
재생률 R: 빠르게, 느리게, 거꾸로

시간 배율은 재생률 R로 표현 → 글로벌 타임라인에 놓는 클립의 길이를 **1/R**배로 늘리거나 줄인다

R = 2 : 두 배 속도



R = −1 : 역재생



4-2 · 글로벌 타임라인

글로벌 시각 τ ↔ 로컬 시각 t 변환

필요한 정보 **시작 시각 τ_start** **재생률 R** **지속 시간 T** **반복 횟수 N**

경우	식
기본	`t = (τ - τ_start)·R , τ = τ_start + t / R`
반복 안 함 (N = 1)	`t = clamp((τ - τ_start)·R, 0, T)`
무한 반복 (N = ∞)	`t = ((τ - τ_start)·R) mod T`
N회 반복	`t = clamp((τ - τ_start)·R, 0, N·T) mod T`

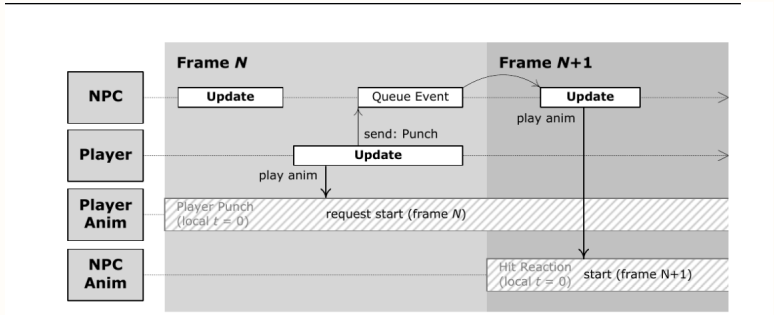
대부분의 엔진은 로컬 타임라인만 직접 다루지만, 글로벌 시간을 쓰면 **애니메이션 동기화가 아주 간단**해진다

어떤 시계로 동기화할까

로컬 시계

클립마다 자기 시계, 재생 시작이 t = 0 → 단순하지만 **동시에 재생해야만** 맞는다

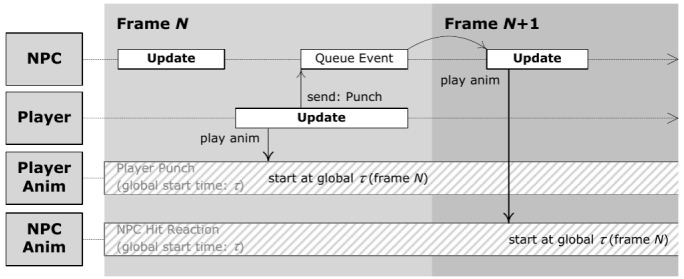
플레이어 펀치 → NPC 피격: 업데이트 순서 때문에 1프레임 지연



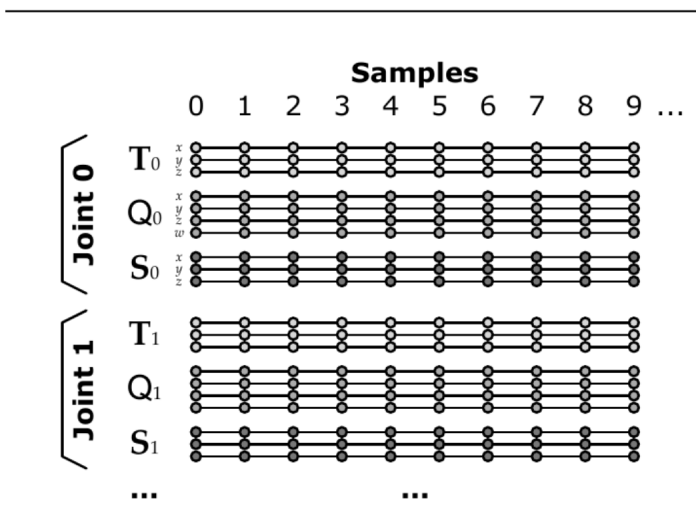
글로벌 시계

클립은 τ_start만 기록 → 시작 시각 값만 같게 하면 **실행 순서와 무관하게** 동기화

피격 반응의 τ_start를 펀치와 같은 값으로 설정



샘플마다, 관절마다 최대 10채널



S (1 또는 3) + Q 쿼터니언 (4) + T (3) = 관절당 최대 **10채널**

```
struct AnimationSample
{
    JointPose* m_aJointPose; // 관절 포즈 배열
};
struct AnimationClip
{
    Skeleton*        m_pSkeleton;
    F32              m_framesPerSecond;
    U32              m_frameCount;
    AnimationSample* m_aSamples; // 샘플 배열
    bool             m_isLooping;
};
```

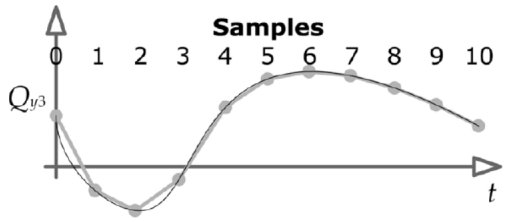
클립은 특정 골격 전용 → m_pSkeleton · 샘플 수: 비반복 frameCount+1, 반복 frameCount · 실제 엔진은 압축해서 저장

연속 채널 함수와 메타채널

연속 채널 함수

샘플은 시간에 대한 연속 함수를 정의한다 (관절당 스칼라 함수 10개)

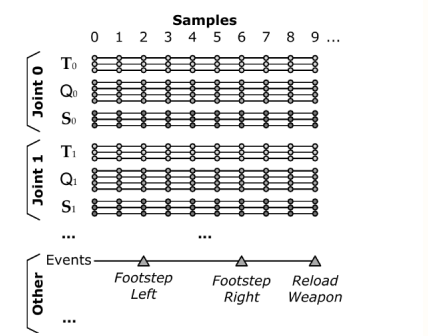
많은 엔진은 샘플 사이를 선형 보간 → **구간별 선형 근사**



메타채널

포즈와 무관하지만 애니메이션과 동기화할 데이터

이벤트 트리거 (발소리·먼지 파티클) · 카메라 로케이터 · 텍스처·머티리얼·조명 파라미터

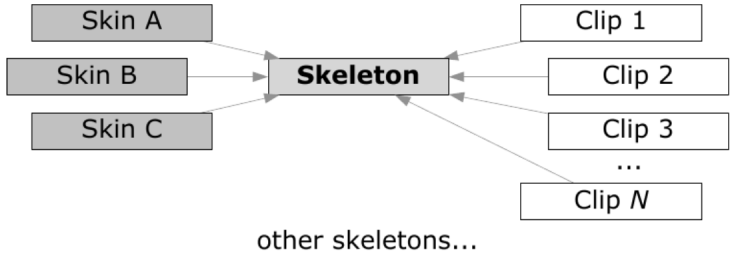


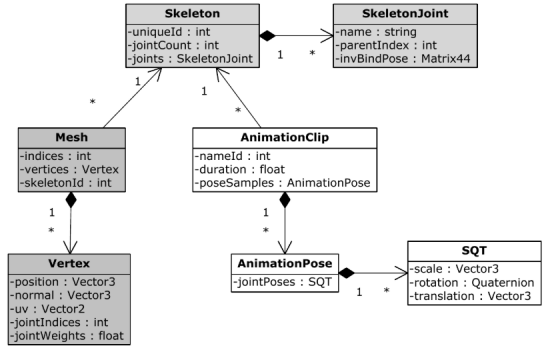
모든 것의 중심은 골격

> 캐릭터 유형 하나 = 골격 1개 + 메시 1개 이상 + 클립 1개 이상

> 스킨은 골격에 붙지만 클립을 모르고, 클립은 골격을 대상으로 하지만 메시를 모른다

> 골격마다 클립 세트가 새로 필요 → 고유 골격 수를 줄이고, 여러 메시를 같은 골격에 스키닝





4-7 · 애니메이션 리타기팅

다른 골격에 애니메이션 적용하기

골격 구조가 같을 때

관절 인덱스를 다시 매핑하는 것만으로 충분할 수 있다

구조가 다를 때 (Naughty Dog)

두 골격 바인드 포즈의 차이를 담은 **리타깃 포즈**로 소스 포즈를 보정

더 고급 기법: Dutreve 등 「Feature Points Based Facial Animation Retargeting」, Hecker 등 「Real-time Motion Retargeting to Highly Varied User-Created Morphologies」

SUMMARY

오늘의 핵심

애니메이션의 종류	Cel → Rigid → Per-Vertex · Morph → Skinned, 자연스러움과 비용 사이의 발전
Skeletons	부모 인덱스를 가진 관절 배열 · 부모가 항상 자식보다 먼저, Index 0 = Root
Poses	Bind(기준 자세) · Local(부모 기준, SRT) · Global(모델·월드 기준, Root까지 곱)
Clips	연속·배율 조절 가능한 로컬 타임라인 · 글로벌 시계로 동기화 · 관절당 10채널 + 메타채널

다음 주제 → **5. Skinning and Matrix Palette Generation**